

Carlos Coronel • Steven Morris

Database Systems

Design, Implementation, & Management



National Economics University

Faculty of Mathematical Economics

Database System Design, Implementation And Management Course – The Relation Database Model

- ❑ Instructor: Duc Minh Vu (MFE)
- ❑ Email: minhvd [at] neu.edu.vn



Learning Objectives

1. Describe the relational database model's logical structure
2. Identify the relational model's basic components and explain the structure, contents, and characteristics of a relational table
3. Use relational database operators to manipulate relational table contents
4. Explain the purpose and components of the data dictionary and system catalog
5. Identify appropriate entities and then the relationships among the entities in the relational database model
6. Describe how data redundancy is handled in the relational database model
7. Explain the purpose of indexing in a relational database



Introduction

- ❑ Good decisions require good information derived from raw facts
- ❑ Data is managed most efficiently when stored in a database
- ❑ Databases evolved from computer file systems
- ❑ Understanding file system characteristics is important



predicate logic

Used extensively in mathematics to provide a framework in which an assertion (statement of fact) can be verified as either true or false.

set theory

A part of mathematical science that deals with sets, or groups of things, and is used as the basis for data manipulation in the relational model.

Note

The relational model, introduced by E. F. Codd in 1970, is based on predicate logic and set theory. **Predicate logic**, used extensively in mathematics, provides a framework in which an assertion (statement of fact) can be verified as either true or false. For example, suppose that a student with a student ID of 12345678 is named Melissa Sanduski. This assertion can easily be demonstrated to be true or false. **Set theory** is a mathematical science that deals with sets, or groups of things, and is used as the basis for data manipulation in the relational model. For example, assume that set A contains three numbers: 16, 24, and 77. This set is represented as $A(16, 24, 77)$. Furthermore, set B contains four numbers, 44, 77, 90, and 11, and so is represented as $B(44, 77, 90, 11)$. Given this information, you can conclude that the intersection of A and B yields a result set with a single number, 77. This result can be expressed as $A \cap B = 77$. In other words, A and B share a common value, 77.

Based on these concepts, the relational model has three well-defined components:

1. A logical data structure represented by relations (see Sections 3-1, 3-2, and 3-5)
2. A set of integrity rules to enforce that the data is consistent and remains consistent over time (see Sections 3-3, 3-6, 3-7, and 3-8)
3. A set of operations that defines how data is manipulated (see Section 3-4)



3-1 A Logical View of Data

- ❑ Database stores and manages both data and metadata.
- ❑ DBMS manages and controls access to the data and the database structure.
- ❑ Such an arrangement—placing the DBMS between the application and the database—eliminates most of the file system's inherent limitations. The result of such flexibility, however, is a far more complex physical structure.
- ❑ Database structures required by both the hierarchical and network database models often become complicated enough to diminish efficient database design.
- ❑ The relational data model allows the designer to focus on the logical representation of the data and its relationships, rather than on the physical storage details.
- ❑ The relational model enables you to view data *logically* rather than *physically*.



3-1a Tables and Their Characteristics

- ❑ A table is perceived as a two-dimensional structure composed of rows and columns.
- ❑ A table is also called a relation because the relational model's creator, E. F. Codd, used the two terms as synonyms.
- ❑ Think of a table as a persistent representation of a logical relation—that is, a relation whose contents can be permanently saved for future use.
- ❑ A table contains a group of related entity occurrences—that is, an entity set
- ❑ The terms entity set and table are often used interchangeably



tuple

In the relational model, a table row.

attribute domain

In data modeling, the construct used to organize and describe an attribute's set of possible values.

Table 3.1 Characteristics of a Relational Table

1	A table is perceived as a two-dimensional structure composed of rows and columns.
2	Each table row (tuple) represents a single entity occurrence within the entity set.
3	Each table column represents an attribute, and each column has a distinct name.
4	Each intersection of a row and column represents a single data value.
5	All values in a column must conform to the same data format.
6	Each column has a specific range of values known as the attribute domain .
7	The order of the rows and columns is immaterial to the DBMS.
8	Each table must have an attribute or combination of attributes that uniquely identifies each row.

Figure 3.1 STUDENT Table Attribute Values

Table name: STUDENT

Database name: Ch03_TinyCollege

STU_NUM	STU_LNAME	STU_FNAME	STU_INIT	STU_DOB	STU_HRS	STU_CLASS	STU_GPA	STU_TRANSFER	DEPT_CODE	STU_PHONE	PROF_NUM
321452	Bowser	William	C	12-Feb-2000	42	So	2.84	No	BIOL	2134	205
324257	Smithson	Anne	K	15-Nov-2001	81	Jr	3.27	Yes	CIS	2256	222
324258	Brewer	Juliette		23-Aug-1999	36	So	2.26	Yes	ACCT	2256	228
324269	Oblonski	Walter	H	16-Sep-1996	66	Jr	3.09	No	CIS	2114	222
324273	Smith	John	D	30-Dec-1988	102	Sr	2.11	Yes	ENGL	2231	199
324274	Katinga	Raphael	P	21-Oct-2000	114	Sr	3.15	No	ACCT	2267	228
324291	Robertson	Gerald	T	08-Apr-1999	120	Sr	3.87	No	EDU	2267	311
324299	Smith	John	B	30-Nov-2000	15	Fr	2.92	No	ACCT	2315	230

STU_NUM = Student number
STU_LNAME = Student last name
STU_FNAME = Student first name
STU_INIT = Student middle initial
STU_DOB = Student date of birth
STU_HRS = Credit hours earned
STU_CLASS = Student classification
STU_GPA = Grade point average
STU_TRANSFER = Student transferred from another institution
DEPT_CODE = Department code
STU_PHONE = 4-digit campus phone extension
PROF_NUM = Number of the professor who is the student's advisor



3-2 Keys

- A key consists of one or more attributes that determine other attributes

domain

Also known as an “attribute domain.” It is the set of allowable values for an attribute.

primary key (PK)

In the relational model, an identifier composed of one or more attributes that uniquely identifies a row. Also, a candidate key selected as a unique entity identifier. See also *key*.

key

One or more attributes that determine other attributes.

See also *candidate key*, *foreign key*, *primary key (PK)*, *secondary key*, and *superkey*.

3-2a Dependencies

- ❑ Determination is the state in which knowing the value of one attribute makes it possible to determine the value of another.
- ❑ Functional dependence: the value of one or more attributes determines the value of one or more other attributes.
- ❑ Key - Determinant: Attribute whose value determines another
- ❑ Dependent: Attribute whose value is determined by the other attribute

functional dependence

Within a relation R, an attribute B is functionally dependent on an attribute A if and only if a given value of attribute A determines exactly one value of attribute B. The relationship "B is dependent on A" is equivalent to "A determines B" and is written as $A \rightarrow B$.

determinant

Any attribute in a specific row whose value directly determines other values in that row. See also *Boyce-Codd normal form (BCNF)*.

dependent

An attribute whose value is determined by another attribute.

determination

The role of a key. In the context of a database table, the statement "A determines B" indicates that knowing the value of attribute A means that the value of attribute B can be looked up.

full functional dependence

A condition in which an attribute is functionally dependent on a composite key but not on any subset of the key.



3-2b Types of Keys

- ❑ A composite key is a key that is composed of more than one attribute.
- ❑ An attribute that is a part of a key is called a key attribute.
- ❑ A superkey is a key that can uniquely identify any row in the table
- ❑ A candidate key is a minimal superkey—that is, a superkey without any unnecessary attributes.
- ❑ Entity integrity is the condition in which each row (entity instance) in the table has its own known, unique identity

entity integrity

The property of a relational table that guarantees each entity has a unique value in a primary key and that the key has no null values.

null

The absence of an attribute value. Note that a null is not a blank.

Figure 3.2 An Example of a Simple Relational Database

Table name: PRODUCT
Primary key: PROD_CODE
Foreign key: VEND_CODE

PROD_CODE	PROD_DESCRIPT	PROD_PRICE	PROD_ON_HAND	VEND_CODE
001278-AB	Claw hammer	12.95	23	232
123-21UUY	Houselite chain saw, 16-in. bar	189.99	4	235
QER-34256	Sledge hammer, 16-lb. head	18.63	6	231
SRE-657UG	Rat-tail file	2.99	15	232
ZZX/3245Q	Steel tape, 12-ft. length	6.79	8	235

Database name: Ch03_SaleCo

Table name: VENDOR
Primary key: VEND_CODE
Foreign key: none

VEND_CODE	VEND_CONTACT	VEND_AREACODE	VEND_PHONE
230	Shelly K. Smithson	608	555-1234
231	James Johnson	615	123-4536
232	Annelise Crystall	608	224-2134
233	Candice Wallace	904	342-6567
234	Arthur Jones	615	123-3324
235	Henry Ortozo	615	899-3425

link



3-2b Types of Keys

- ❑ A foreign key (FK) is the primary key of one table that has been placed into another table to create a common attribute.
- ❑ Foreign keys are used to ensure referential integrity, the condition in which every reference to an entity instance by another entity instance is valid.
- ❑ A secondary key is defined as a key that is used strictly for data retrieval purposes and does not require a functional dependency

referential integrity

A condition by which a dependent table's foreign key entry must have either a null entry or a matching entry in the primary key of the related table.

secondary key

A key used strictly for data retrieval purposes that does not require a functional dependency. For example, customers are not likely to know their customer number (primary key), but the combination of last name, first name, middle initial, and telephone number will probably match the appropriate table row. See also *key*.



Table 3.3 Relational Database Keys

Key Type	Definition
Superkey	An attribute or combination of attributes that uniquely identifies each row in a table
Candidate key	A minimal (irreducible) superkey; a superkey that does not contain a subset of attributes that is itself a superkey
Primary key	A candidate key selected to uniquely identify all other attribute values in any given row; cannot contain null entries
Foreign key	An attribute or combination of attributes in one table whose values must either match the primary key in another table or be null
Secondary key	An attribute or combination of attributes used strictly for data retrieval purposes

3-3 Integrity Rules

Table 3.4 Integrity Rules

Entity Integrity	Description
Requirement	All primary key entries are unique, and no part of a primary key may be null.
Purpose	Each row will have a known, unique identity, and foreign key values can properly reference primary key values.
Example	No invoice can have a duplicate number, nor can it be null; in short, all invoices are uniquely identified by their invoice number.
Referential Integrity	Description
Requirement	A foreign key may have either a null entry, as long as it is not a part of its table's primary key, or an entry that matches the primary key value in a table to which it is related (every non-null foreign key value <i>must</i> reference an <i>existing</i> primary key value).
Purpose	The purpose is to ensure that every reference by a foreign key is a valid reference to the related primary key. It is possible for an attribute <i>not</i> to have a corresponding value, but it will be impossible to have an invalid entry; the enforcement of the referential integrity rule makes it impossible to delete a row in one table whose primary key has mandatory matching foreign key values in another table.
Example	A customer might not yet have an assigned sales representative (number), but it will be impossible to have an invalid sales representative (number).

Figure 3.3 An Illustration of Integrity Rules

Table name: CUSTOMER
Primary key: CUS_CODE
Foreign key: AGENT_CODE

Database name: Ch03_InsureCo

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_RENEW_DATE	AGENT_CODE
10010	Ramas	Alfred	A	05-Apr-2024	502
10011	Dunne	Leona	K	16-Jun-2024	501
10012	Smith	Kathy	W	29-Jan-2025	502
10013	Olowski	Paul	F	14-Oct-2024	
10014	Orlando	Myron		28-Dec-2024	501
10015	O'Brian	Amy	B	22-Sep-2024	503
10016	Brown	James	G	25-Mar-2025	502
10017	Williams	George		17-Jul-2024	503
10018	Farriss	Anne	G	03-Dec-2024	501
10019	Smith	Olette	K	14-Mar-2025	503

Table name: AGENT (only five selected fields are shown)
Primary key: AGENT_CODE
Foreign key: none

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
501	713	228-1249	Alby	132735.75
502	615	882-1244	Hahn	138967.35
503	615	123-5589	Okon	127093.45



3-3 Integrity Rules (cont'd.)

Entity integrity.

- The CUSTOMER table's primary key is CUS_CODE. The CUSTOMER primary key column has no null entries, and all entries are unique.
- The AGENT table's primary key is AGENT_CODE, and this primary key column is also free of null entries.

Referential integrity.

- The CUSTOMER table contains a foreign key, AGENT_CODE, that links entries in the CUSTOMER table to the AGENT table.
- The CUS_CODE row identified by the (primary key) number 10013 contains a null entry in its AGENT_CODE foreign key because Paul F. Olowski does not yet have a sales representative assigned to him. The remaining AGENT_CODE entries in the CUSTOMER table all match the AGENT_CODE entries in the AGENT table.



flags

Special codes implemented by designers to trigger a required response, alert end users to specified conditions, or encode values. Flags may be used to prevent nulls by bringing attention to the absence of a value in a table.

Table 3.5 A Dummy Variable Value Used as a Flag

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
-99	000	000-0000	None	\$0.00

To avoid nulls, some designers use special codes, known as flags, to indicate the absence of some value



3-4 Relational Algebra

□ **Relational algebra** defines the theoretical way of manipulating table contents using relational operators.

relational algebra

A set of mathematical principles that form the basis for manipulating relational table contents; the eight main functions are SELECT, PROJECT, JOIN, INTERSECT, UNION, DIFFERENCE, PRODUCT, and DIVIDE.

Note

The degree of relational completeness can be defined by the extent to which relational algebra is supported. To be considered minimally relational, the DBMS must support the key relational operators SELECT, PROJECT, and JOIN.



3-4a Formal Definitions and Terminology

- ❑ A relation is the data in your tables.
- ❑ A **relvar** is a variable that holds a relation.
- ❑ A relvar has two parts: the heading and the body.
 - The relvar heading contains the names of the attributes, while the relvar body contains the relation.
 - To conveniently maintain this distinction in formulas, an unspecified relation is often assigned a lowercase letter (e.g., “ r ”), while the relvar is assigned an uppercase letter (e.g., “ R ”).
 - You could then say that r is a relation of type R , or $r(R)$

relvar

Short for relation variable, a variable that holds a relation. A relvar is a container (variable) for holding relation data, not the relation itself.



3-4b Relational Set Operators

□ The relational operators have the property of **closure**;

➤ The use of relational algebra operators on existing relations (tables)
produces new relations

□ the focus will be on the SELECT (or RESTRICT),
PROJECT, UNION, INTERSECT, DIFFERENCE, PRODUCT,
JOIN, and DIVIDE operators.

closure

A property of relational operators that permits the use of relational algebra operators on existing tables (relations) to produce new relations.



3-4b Relational Set Operators

- ❑ **SELECT**, also known as **RESTRICT**, yields values for all rows found in the table that satisfy a given condition.
- ❑ **PROJECT** yields all values for selected attributes. PROJECT yields a vertical subset of a table.
- ❑ **UNION** combines all rows from two tables, excluding duplicate rows. The tables must have the same attribute characteristics (**union-compatible**).
- ❑ **INTERSECT** yields only the rows that appear in both tables. The tables must be union-compatible.
- ❑ **DIFFERENCE (subtraction)** yields all rows in one table that are not found in the other table.
- ❑ **PRODUCT** yields all possible pairs of rows from two tables—also known as the Cartesian product.



SELECT

In relational algebra, an operator used to select a subset of rows. Also known as *RESTRICT*.

RESTRICT

See *SELECT*.

PROJECT

In relational algebra, an operator used to select a subset of columns.

UNION

In relational algebra, an operator used to merge (append) two tables into a new table, dropping the duplicate rows. The tables must be *union-compatible*.

union-compatible

Two or more tables that have the same number of columns and the corresponding columns have compatible domains.

INTERSECT

In relational algebra, an operator used to yield only the rows that are common to two union-compatible tables.

DIFFERENCE

In relational algebra, an operator used to yield all rows from one table that are not found in another union-compatible table.

Figure 3.4 Select**Original table**

P_CODE	P_DESCRIPTION	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

SELECT ALL yields**New table**

P_CODE	P_DESCRIPTION	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

SELECT only PRICE less than \$2.00 yields

P_CODE	P_DESCRIPTION	PRICE
213345	9v battery	1.92
254467	100W bulb	1.47

SELECT only P_CODE = 311452 yields

P_CODE	P_DESCRIPTION	PRICE
311452	Powerdrill	34.99



Note

Formally, SELECT is denoted by the lowercase Greek letter sigma (σ). Sigma is followed by the condition to be evaluated (called a predicate) as a subscript, and then the relation is listed in parentheses. For example, to SELECT all of the rows in the CUSTOMER table that have the value "10010" in the CUS_CODE attribute, you would write the following:

$$\sigma_{\text{cus_code} = 10010}(\text{customer})$$

Figure 3.5 Project

Original table

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

PROJECT PRICE yields

New table

PRICE
5.26
25.15
10.99
1.92
1.47
34.99

PROJECT P_DESCRIPT and PRICE yields

P_DESCRIPT	PRICE
Flashlight	5.26
Lamp	25.15
Box Fan	10.99
9v battery	1.92
100W bulb	1.47
Powerdrill	34.99

PROJECT P_CODE and PRICE yields

P_CODE	PRICE
123456	5.26
123457	25.15
123458	10.99
213345	1.92
254467	1.47
311452	34.99

Note

Formally, PROJECT is denoted by the Greek letter pi (π). Some sources use the uppercase letter, and other sources use the lowercase letter. Codd used the lowercase π in his original article on the relational model, and that is what we use here. Pi is followed by the list of attributes to be returned as subscripts and then the relation listed in parentheses. For example, to PROJECT the CUS_FNAME and CUS_LNAME attributes in the CUSTOMER table, you would write the following:

$$\pi_{\text{cus_fname, cus_lname}} (\text{customer})$$

Because relational operators have the property of closure, that is, they accept relations as input and produce relations as output, it is possible to combine operators. For example, you can combine the two previous operators to find the first and last name of the customer with customer code 10010:

$$\pi_{\text{cus_fname, cus_lname}} (\sigma_{\text{cus_code} = 10010} (\text{customer}))$$

Figure 3.6 Union

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

UNION

P_CODE	P_DESCRIPT	PRICE
345678	Microwave	160.00
345679	Dishwasher	500.00
123458	Box Fan	10.99

yields



P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99
345678	Microwave	160
345679	Dishwasher	500



Note

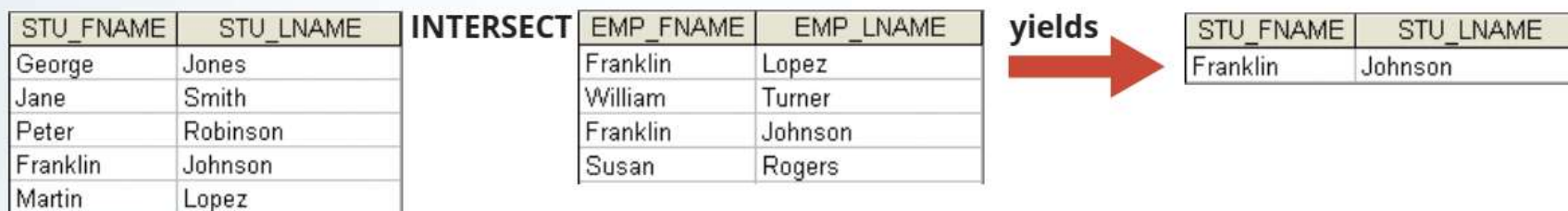
UNION is denoted by the symbol $\cup$. If the relations SUPPLIER and VENDOR are union-compatible, then a UNION between them would be denoted as follows:

$\text{supplier} \cup \text{vendor}$

It is rather unusual to find two relations that are union-compatible in a database. Typically, PROJECT operators are applied to relations to produce results that are union-compatible. For example, assume the SUPPLIER and VENDOR tables are not union-compatible. If you want to produce a listing of all vendor and supplier names, then you can PROJECT the names from each table and then perform a UNION with them.

$\pi_{\text{supplier_name}}(\text{supplier}) \cup \pi_{\text{vendor_name}}(\text{vendor})$

Figure 3.7 Intersect





Note

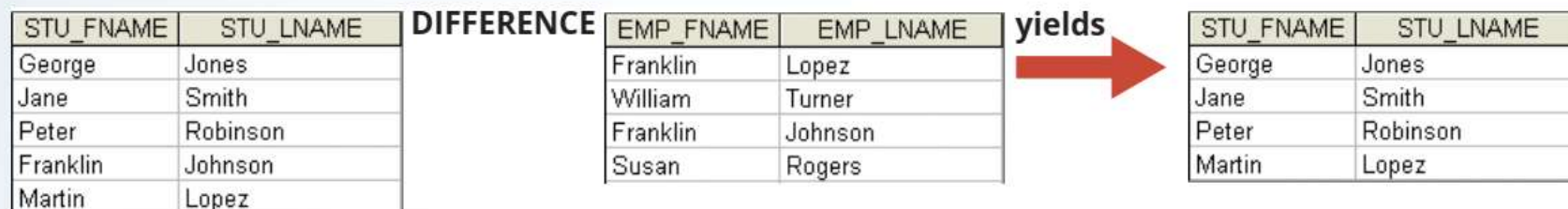
INTERSECT is denoted by the symbol $\cap$. If the relations SUPPLIER and VENDOR are union-compatible, then an INTERSECT between them would be denoted as follows:

$\text{supplier} \cap \text{vendor}$

Just as with the UNION operator, it is unusual to find two relations that are union-compatible in a database, so PROJECT operators are applied to relations to produce results that can be manipulated with an INTERSECT operator. For example, again assume the SUPPLIER and VENDOR tables are not union-compatible. If you want to produce a listing of any vendor and supplier names that are the same in both tables, then you can PROJECT the names from each table and then perform an INTERSECT with them.

$\pi_{\text{supplier_name}}(\text{supplier}) \cap \pi_{\text{vendor_name}}(\text{vendor})$

Figure 3.8 Difference





Note

DIFFERENCE is denoted by the minus symbol $-$. If the relations SUPPLIER and VENDOR are union-compatible, then a DIFFERENCE of SUPPLIER minus VENDOR would be written as follows:

supplier $-$ vendor

Assuming the SUPPLIER and VENDOR tables are not union-compatible, producing a list of any supplier names that do not appear as vendor names, then you can use a DIFFERENCE operator.

$\pi_{\text{supplier_name}}(\text{supplier}) - \pi_{\text{vendor_name}}(\text{vendor})$

Figure 3.9 Product

P_CODE	P_DESCRIPT	PRICE
123456	Flashlight	5.26
123457	Lamp	25.15
123458	Box Fan	10.99
213345	9v battery	1.92
254467	100W bulb	1.47
311452	Powerdrill	34.99

PRODUCT

STORE	aisle	shelf
23	W	5
24	K	9
25	Z	6

yields



P_CODE	P_DESCRIPT	PRICE	STORE	aisle	shelf
123456	Flashlight	5.26	23	W	5
123456	Flashlight	5.26	24	K	9
123456	Flashlight	5.26	25	Z	6
123457	Lamp	25.15	23	W	5
123457	Lamp	25.15	24	K	9
123457	Lamp	25.15	25	Z	6
123458	Box Fan	10.99	23	W	5
123458	Box Fan	10.99	24	K	9
123458	Box Fan	10.99	25	Z	6
213345	9v battery	1.92	23	W	5
213345	9v battery	1.92	24	K	9
213345	9v battery	1.92	25	Z	6
311452	Powerdrill	34.99	23	W	5
311452	Powerdrill	34.99	24	K	9
311452	Powerdrill	34.99	25	Z	6
254467	100W bulb	1.47	23	W	5
254467	100W bulb	1.47	24	K	9
254467	100W bulb	1.47	25	Z	6



Note

PRODUCT is denoted by the multiplication symbol $\times$. The PRODUCT of the CUSTOMER and AGENT relations would be written as follows:

customer $\times$ agent

A Cartesian product produces a set of sequences in which every member of one set is paired with every member of another set. In terms of relations, this means that every tuple in one relation is paired with every tuple in the second relation.

3-4b Relational Set Operators

□ **JOIN** allows information to be intelligently combined from two or more tables.

- A **natural join** links tables by selecting only the rows with common values in their common attribute(s).
- **Equijoin**, links tables on the basis of an equality condition that compares specified columns of each table. The outcome of the equijoin does not eliminate duplicate columns.
- If any other comparison operator is used, the join is called a **theta join**.

JOIN

In relational algebra, a type of operator used to yield rows from two tables based on criteria. There are many types of joins, such as natural join, theta join, equijoin, and outer join.

natural join

A relational operation that yields a new table composed of only the rows with common values in their common attribute(s).

equijoin

A join operator that links tables based on an equality condition that compares specified columns of the tables.

theta join

A join operator that links tables using an inequality comparison operator ($<$, $>$, $<=$, $>=$) in the join condition.



Note

Natural join is normally just referred to as JOIN in formal treatments. JOIN is denoted by the symbol $\bowtie$. The JOIN of the CUSTOMER and AGENT relations would be written as follows:

customer $\bowtie$ agent

Notice that the JOIN of two relations returns all of the attributes of both relations, except only one copy of the common attribute is returned. Formally, this is described as a UNION of the relvar headings. Therefore, the JOIN of the relations ($c \bowtie a$) includes the UNION of the relvars ($C \cup A$). Also note that, as described earlier, JOIN is not a fundamental relational algebra operator. It can be derived from other operators as follows:

$\pi_{\text{cus_code, cus_lname, cus_fname, cus_initial, cus_renew_date, agent_code, agent_areacode, agent_phone, agent_lname, agent_ytd_sls}}$
 $(\sigma_{\text{customer.agent_code} = \text{agent.agent_code}} (\text{customer} \times \text{agent}))$



Note

In formal terms, theta join is considered an extension of natural join. Theta join is denoted by adding a theta subscript after the JOIN symbol: $\bowtie_{\theta}$. Equijoin is then a special type of theta join.

Figure 3.10 Two Tables That Will Be Used in JOIN Illustrations**Table name: CUSTOMER**

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	421
1657399	Vanloo	32145	231

Table name: AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

A **natural join** links tables by selecting only the rows with common values in their common attribute(s). A natural join is the result of a three-stage process:

Figure 3.11 Natural Join, Step 1: PRODUCT

1. First, a PRODUCT of the tables is created, yielding the results shown in Figure 3.11.

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1132445	Walker	32145	231	125	6152439887
1132445	Walker	32145	231	167	6153426778
1132445	Walker	32145	231	231	6152431124
1132445	Walker	32145	231	333	9041234445
1217782	Adares	32145	125	125	6152439887
1217782	Adares	32145	125	167	6153426778
1217782	Adares	32145	125	231	6152431124
1217782	Adares	32145	125	333	9041234445
1312243	Rakowski	34129	167	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1312243	Rakowski	34129	167	231	6152431124
1312243	Rakowski	34129	167	333	9041234445
1321242	Rodriguez	37134	125	125	6152439887
1321242	Rodriguez	37134	125	167	6153426778
1321242	Rodriguez	37134	125	231	6152431124
1321242	Rodriguez	37134	125	333	9041234445
1542311	Smithson	37134	421	125	6152439887
1542311	Smithson	37134	421	167	6153426778
1542311	Smithson	37134	421	231	6152431124
1542311	Smithson	37134	421	333	9041234445
1657399	Vanloo	32145	231	125	6152439887
1657399	Vanloo	32145	231	167	6153426778
1657399	Vanloo	32145	231	231	6152431124
1657399	Vanloo	32145	231	333	9041234445

Figure 3.12 Natural Join, Step 2: SELECT

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124

2. Second, a SELECT is performed on the output of Step 1 to yield only the rows for which the AGENT_CODE values are equal. The common columns are referred to as the **join columns**. Step 2 yields the results shown in Figure 3.12.

join columns

Columns that are used in the criteria of join operations. The join columns generally share similar values.

Figure 3.13 Natural Join, Step 3: PROJECT

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	6152439887
1321242	Rodriguez	37134	125	6152439887
1312243	Rakowski	34129	167	6153426778
1132445	Walker	32145	231	6152431124
1657399	Vanloo	32145	231	6152431124

3. A PROJECT is performed on the results of Step 2 to yield a single copy of each attribute, thereby eliminating duplicate columns. Step 3 yields the output shown in Figure 3.13.



3-4b Relational Set Operators (cont'd.)

- ❑ An **inner join** returns matched records from the tables that are being joined.
- ❑ In an **outer join**, the matched pairs would be retained, and any unmatched values in the other table would be left null.

inner join

A join operation in which only rows that meet a given criterion are selected. The criterion can be an equality condition (natural join or equijoin) or an inequality condition (theta join). The most commonly used type of join.

outer join

A join operation that produces a table in which all unmatched pairs are retained; unmatched values in the related table are left null.

left outer join

A join operation that yields all the rows in the left table, including those that have no matching values in the other table.

right outer join

A join operation that yields all of the rows in the right table, including the ones with no matching values in the other table.

Figure 3.14 Left Outer Join

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124
1542311	Smithson	37134	421		

A **left outer join** yields all of the rows in the CUSTOMER table, including those that do not have a matching value in the AGENT table. An example of such a join is shown in Figure 3.14

Figure 3.15 Right Outer Join

CUS_CODE	CUS_LNAME	CUS_ZIP	CUSTOMER.AGENT_CODE	AGENT.AGENT_CODE	AGENT_PHONE
1217782	Adares	32145	125	125	6152439887
1321242	Rodriguez	37134	125	125	6152439887
1312243	Rakowski	34129	167	167	6153426778
1132445	Walker	32145	231	231	6152431124
1657399	Vanloo	32145	231	231	6152431124
				333	9041234445

A **right outer join** yields all of the rows in the AGENT table, including those that do not have matching values in the CUSTOMER table. An example of such a join is shown in Figure 3.15.



3-4b Relational Set Operators (cont'd.)

- ❑ The **DIVIDE** operator is used to answer questions about one set of data being associated with all values of data in another set of data.
- ❑ The DIVIDE operation uses one double-column table (Table 1) as the dividend and one single-column table (Table 2) as the divisor.

DIVIDE

In relational algebra, an operator that answers queries about one set of data being associated with all values of data in another set of data.

Note

The DIVIDE operator is denoted by the division symbol $\div$. Given two relations, R and S, the DIVISION of them would be written as $r \div s$.

Figure 3.16 Divide

P_CODE	CUS_CODE
123456	10400
123456	11501
123456	10030
123456	12550
234567	12350
234567	10040
234567	10500
234567	10030
234567	12550
345678	10400
345678	11630
345678	12550
456789	11630
567890	10600
567890	10030
567890	12550
678901	11500
678901	10400
678901	11630

DIVIDE

P_CODE
123456
234567
567890

yields

CUS_CODE
10030
12550



3-5 The Data Dictionary and the System Catalog

❑ **Data dictionary** provides a detailed description of all tables in the database.

- Contains at least all of the attribute names and characteristics for each table in the system. The data dictionary contains metadata—data about data.

❑ **System catalog** can be described as a detailed system data dictionary that describes all objects within the database (table names, a table's creator and creation date, number of columns in each table ...)

- System catalog is a system-created database whose tables store the user/designer-created database characteristics and contents.
- Can be queried just like any user/designer created table.

data dictionary

A DBMS component that stores metadata—data about data. Thus, the data dictionary contains the data definition as well as their characteristics and relationships. A data dictionary may also include data that are external to the DBMS. Also known as an *information resource dictionary*. See also *active data dictionary*, *metadata*, and *passive data dictionary*.

system catalog

A detailed system data dictionary that describes all objects in a database.



3-5 The Data Dictionary and the System Catalog

- ❑ System catalog automatically produces database documentation.
- ❑ As new tables are added to the database, that documentation also allows the RDBMS to check for and eliminate homonyms and synonyms.
- ❑ The word *homonym* indicates the use of the same name to label different attributes.
- ❑ a **synonym** is the opposite of a homonym and indicates the use of different names to describe the same attribute.

homonym

The use of the same name to label different attributes. Homonyms generally should be avoided. See also *synonym*.

synonym

The use of different names to identify the same object, such as an entity, an attribute, or a relationship; synonyms should generally be avoided. See also *homonym*.

3-6 Relationships within the Relational Database

- The one-to-many (1:M) relationship is easily implemented in the relational model by putting the *primary key of the “1” side into the table of the “many” side as a foreign key*.

Figure 3.18 The Implemented 1:M Relationship between PAINTER and PAINTING

Table name: PAINTER
Primary key: PAINTER_NUM
Foreign key: none

Database name: Ch03_Museum

PAINTER_NUM	PAINTER_LNAME	PAINTER_FNAME	PAINTER_INITIAL
123	Ross	Georgette	P
126	Ittero	Julio	G

Table name: PAINTING
Primary key: PAINTING_NUM
Foreign key: PAINTER_NUM

PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126



3-6 Relationships within the Relational Database

- One entity in a 1:1 relationship can be related to only one other entity, and vice versa.
 - one department chair—a professor—can chair only one department, and one department can have only one department chair

Figure 3.21 The 1:1 Relationship between PROFESSOR and DEPARTMENT

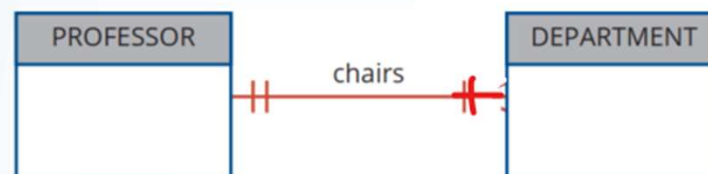


Table name: PROFESSOR
Primary key: EMP_NUM
Foreign key: DEPT_CODE

Database name: Ch03_TinyCollege

EMP_NUM	DEPT_CODE	PROF_OFFICE	PROF_EXTENSION	PROF_HIGH_DEGREE
103	HIST	DRE 156	6783	Ph.D.
104	ENG	DRE 102	5561	MA
105	ACCT	KLR 229D	8665	Ph.D.
106	MKT/MGT	KLR 126	3899	Ph.D.
110	BIOL	AAK 160	3412	Ph.D.
114	ACCT	KLR 211	4436	Ph.D.
155	MATH	AAK 201	4440	Ph.D.
160	ENG	DRE 102	2248	Ph.D.
162	CIS	KLR 203E	2359	Ph.D.
191	MKT/MGT	KLR 409B	4016	DBA
195	PSYCH	AAK 297	3550	Ph.D.
209	CIS	KLR 333	3421	Ph.D.
228	CIS	KLR 300	3000	Ph.D.
297	MATH	AAK 194	1145	Ph.D.
299	ECON/FIN	KLR 284	2851	Ph.D.
301	ACCT	KLR 244	4683	Ph.D.
335	ENG	DRE 208	2000	Ph.D.
342	SOC	BBG 208	5514	Ph.D.
387	BIOL	AAK 230	8665	Ph.D.
401	HIST	DRE 156	6783	MA
425	ECON/FIN	KLR 284	2851	MBA
435	ART	BBG 185	2278	Ph.D.

Each professor belongs to a department



The 1:M DEPARTMENT employs PROFESSOR relationship is implemented through the placement of the DEPT_CODE foreign key in the PROFESSOR table.



Table name: DEPARTMENT
Primary key: DEPT_CODE
Foreign key: EMP_NUM



The 1:1 PROFESSOR chairs DEPARTMENT relationship is implemented through the placement of the EMP_NUM foreign key in the DEPARTMENT table.

DEPT_CODE	DEPT_NAME	SCHOOL_CODE	EMP_NUM	DEPT_ADDRESS	DEPT_EXTENSION
ACCT	Accounting	BUS	114	KLR 211, Box 52	3119
ART	Fine Arts	A&SCI	435	BBG 185, Box 128	2278
BIOL	Biology	A&SCI	387	AAK 230, Box 415	4117
CIS	Computer Info. Systems	BUS	209	KLR 333, Box 56	3245
ECON/FIN	Economics/Finance	BUS	299	KLR 284, Box 63	3126
ENG	English	A&SCI	160	DRE 102, Box 223	1004
HIST	History	A&SCI	103	DRE 156, Box 284	1867
MATH	Mathematics	A&SCI	297	AAK 194, Box 422	4234
MKT/MGT	Marketing/Management	BUS	106	KLR 126, Box 55	3342
PSYCH	Psychology	A&SCI	195	AAK 297, Box 438	4110
SOC	Sociology	A&SCI	342	BBG 208, Box 132	2008

Each department is chaired by a professor.

So 1:1?

3-6c The M:N Relationship

- ❑ A many-to-many (M:N) relationship is not supported directly in the relational environment.
- ❑ M:N relationships can be implemented by creating a new entity in 1:M relationships with the original entities

Figure 3.23 The ERM's M:N Relationship between STUDENT and CLASS

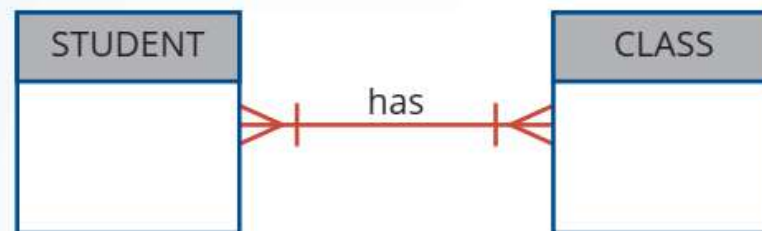


Figure 3.24 The Wrong Implementation of the M:N Relationship between STUDENT and CLASS

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

Database name: Ch03_CollegeTry

STU_NUM	STU_LNAME	CLASS_CODE
321452	Bowser	10014
321452	Bowser	10018
321452	Bowser	10021
324257	Smithson	10014
324257	Smithson	10018
324257	Smithson	10021

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: STU_NUM

CLASS_CODE	STU_NUM	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	321452	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10014	324257	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	321452	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10018	324257	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10021	321452	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114
10021	324257	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114

Figure 3.25 Converting the M:N Relationship into Two 1:M Relationships

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

STU_NUM	STU_LNAME
321452	Bowser
324257	Smithson

Table name: ENROLL
Primary key: CLASS_CODE + STU_NUM
Foreign key: CLASS_CODE, STU_NUM

CLASS_CODE	STU_NUM	ENROLL_GRADE
10014	321452	C
10014	324257	B
10018	321452	A
10018	324257	B
10021	321452	C
10021	324257	C

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10021	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114

Database name: Ch03_CollegeTry2

composite entity

An entity designed to transform an M:N relationship into two 1:M relationships. The composite entity's primary key comprises at least the primary keys of the entities that it connects. Also known as a *bridge entity* or *associative entity*. See also *linking table*.

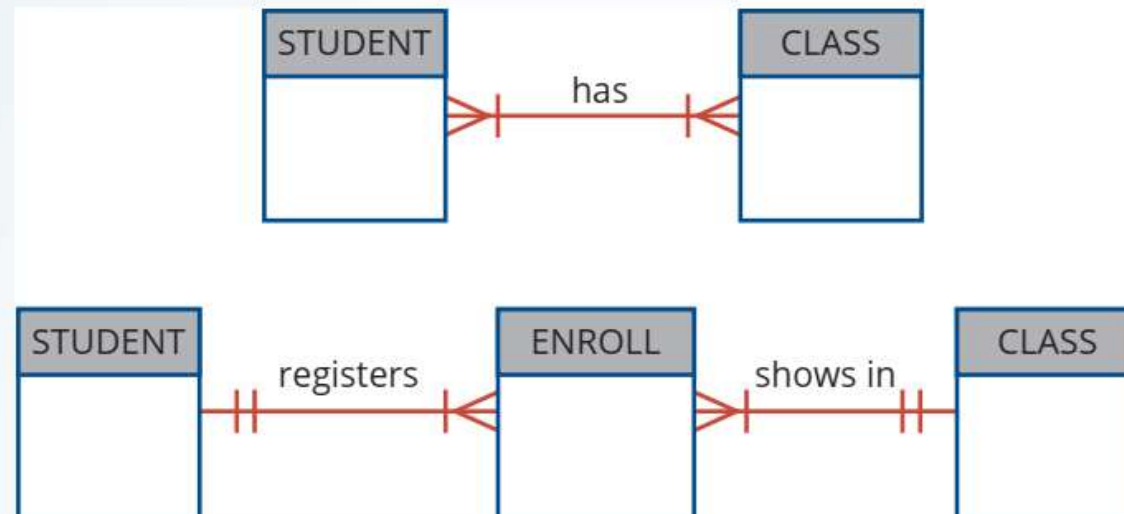
bridge entity

See *composite entity*.

associative entity

See *composite entity*.

Figure 3.26 Changing the M:N Relationships to Two 1:M Relationships



3-9 Codd's Relational Database Rules

Table 13.8 Dr. Codd's 12 Relational Database Rules

Rule	Rule Name	Description
1	Information	All information in a relational database must be logically represented as column values in rows within tables.
2	Guaranteed access	Every value in a table is guaranteed to be accessible through a combination of table name, primary key value, and column name.
3	Systematic treatment of nulls	Nulls must be represented and treated in a systematic way, independent of data type.
4	Dynamic online catalog based on the relational model	The metadata must be stored and managed as ordinary data—that is, in tables within the database; such data must be available to authorized users using the standard database relational language.
5	Comprehensive data sublanguage	The relational database may support many languages; however, it must support one well-defined, declarative language as well as data definition, view definition, data manipulation (interactive and by program), integrity constraints, authorization, and transaction management (begin, commit, and rollback).
6	View updating	Any view that is theoretically updatable must be updatable through the system.

3-9 Codd's Relational Database Rules

7	High-level insert, update, and delete	The database must support set-level inserts, updates, and deletes.
8	Physical data independence	Application programs and ad hoc facilities are logically unaffected when physical access methods or storage structures are changed.
9	Logical data independence	Application programs and ad hoc facilities are logically unaffected when changes are made to the table structures that preserve the original table values (changing order of columns or inserting columns).
10	Integrity independence	All relational integrity constraints must be definable in the relational language and stored in the system catalog, not at the application level.
11	Distribution independence	The end users and application programs are unaware of and unaffected by the data location (distributed vs. local databases).
12	Nonsubversion	If the system supports low-level access to the data, users must not be allowed to bypass the integrity rules of the database.
13	Rule zero	All preceding rules are based on the notion that to be considered relational, a database must use its relational facilities exclusively for management.

¹Codd, E., "Is Your DBMS Really Relational?" and "Does Your DBMS Run by the Rules?" *Computerworld*, October 14 and 21, 1985.



Summary

- Tables are the basic building blocks of a relational database. A grouping of related entities, known as an entity set, is stored in a table. Conceptually speaking, the relational table is composed of intersecting rows (tuples) and columns. Each row represents a single entity, and each column represents the characteristics (attributes) of the entities.
- Keys are central to the use of relational tables. Keys define functional dependencies; that is, other attributes are dependent on the key and can therefore be found if the key value is known. A key can be classified as a superkey, a candidate key, a primary key, a secondary key, or a foreign key.
- Each table row must have a primary key. The primary key is an attribute or combination of attributes that uniquely identifies all remaining attributes found in any given row. Because a primary key must be unique, no null values are allowed if entity integrity is to be maintained.



- Although tables are independent, they can be linked by common attributes. Thus, the primary key of one table can appear as the foreign key in another table to which it is linked. Referential integrity dictates that the foreign key must contain values that match the primary key in the related table or must contain nulls.
- The relational model supports several relational algebra functions, including SELECT, PROJECT, JOIN, INTERSECT, UNION, DIFFERENCE, PRODUCT, and DIVIDE. Understanding the basic mathematical forms of these functions gives a broader understanding of the data manipulation options.
- A relational database performs much of the data manipulation work behind the scenes. For example, when you

create a database, the RDBMS automatically produces a structure to house a data dictionary for your database. Each time you create a new table within the database, the RDBMS updates the data dictionary, thereby providing the database documentation.

- Once you know the basics of relational databases, you can concentrate on design. Good design begins by identifying appropriate entities and their attributes and then the relationships among the entities. Those relationships (1:1, 1:M, and M:N) can be represented using ERDs. The use of ERDs allows you to create and evaluate simple logical design. The 1:M relationship is most easily incorporated in a good design; just make sure that the primary key of the “1” is included in the table of the “many.”

Key Terms

associative entity

attribute domain

bridge entity

candidate key

closure

composite entity

composite key

data dictionary

dependent

determinant

determination

DIFFERENCE

DIVIDE

domain

entity integrity

equijoin

flags

foreign key (FK)

full functional dependence

functional dependence

homonym

index

index key

inner join

INTERSECT

JOIN

join columns

key

key attribute

left outer join

linking table

natural join

null

outer join

predicate logic

primary key (PK)

PRODUCT

PROJECT

referential integrity

relational algebra

relvar

RESTRICT

right outer join

secondary key

SELECT

set theory

superkey

synonym

system catalog

theta join

tuple

UNION

union-compatible

unique index